Flight Delay Prediction

36-673 Final Project

Ellie Huang (elliehua) and Nikita Jayaprakash (njayapra)

2026-04-22

Data Prep

Load Libraries and Data

```
# Load data
flights2023 <- read_csv(here("data", "flights2023.csv")) |>
 clean_names()

flights2024 <- read_csv(here("data", "flights2024.csv")) |>
 clean_names()

test <- read_csv(here("data", "flights2025.csv")) |>
 clean_names()

test_visible <- read_csv(here("data", "flights2025_visible.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

test_guess <- read_csv(here("data", "flights2025_guess.csv")) |>
 clean_names() |>
  mutate(data_year = 2025)

# Build training set
train <- bind_rows(
 flights2023 |> mutate(data_year = 2023),
 flights2024 |> mutate(data_year = 2024)
)
```

Clean and Filter Data

```
# Apply the same cleaning steps to each data frame so train and test sets are consistent
clean_flights <- function(df) {
 df |>
 clean_names() |>
 mutate(
 fl_date = as.Date(fl_date),
```

```r
 dep_del15 = as.numeric(dep_del15),
 op_unique_carrier = as.factor(op_unique_carrier),
 origin = as.factor(origin),
 dest = as.factor(dest),
 sched_dep_hour = floor(crs_dep_time / 100),
 month = as.factor(month),
 day_of_week = as.factor(day_of_week)
 )
}

train <- clean_flights(train)
test <- clean_flights(test)
test_visible <- clean_flights(test_visible)
test_guess <- clean_flights(test_guess)
```

```r
# Restrict to departures from PIT
train <- train |>
 filter(origin == "PIT")

test_visible <- test_visible |>
 filter(origin == "PIT")

test_guess <- test_guess |>
 filter(origin == "PIT")

# Quick check of the target distribution
table(train$dep_del15, useNA = "ifany")
```

```
##
##     0     1  <NA>
## 59534 10972   950
```

We focus on Pittsburgh departures because that is the airport represented in the prediction task. We also use `sched_dep_hour` rather than actual departure time, since actual departure time is not available for the flights we need to predict.

Exploratory Data Analysis

The goal of the EDA is not just to describe the data, but to identify patterns that are both predictive and available before departure. In particular, we want to see whether delay risk depends in a stable way on scheduling, carrier, route, seasonality, and same-day operating conditions.

Missingness

```r
# Missingness in the main timing and target fields
colSums(is.na(train[, c("dep_del15", "dep_time", "crs_dep_time")]))
```

```
##    dep_del15     dep_time crs_dep_time
##          950          950            0
```

```r
# Variables with nonzero missingness
train |>
 summarise(across(everything(), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 filter(pct_missing > 0) |>
 arrange(desc(pct_missing)) |>
 print(n = 30)
```

```
## # A tibble: 27 x 2
##    variable            pct_missing
##    <chr>                     <dbl>
##  1 first_dep_time          0.995
##  2 total_add_gtime         0.995
##  3 longest_add_gtime       0.995
##  4 cancellation_code       0.986
##  5 carrier_delay           0.837
##  6 weather_delay           0.837
##  7 nas_delay               0.837
##  8 security_delay          0.837
##  9 late_aircraft_delay     0.837
## 10 fl_date                 0.609
## 11 arr_delay               0.0154
## 12 arr_delay_new           0.0154
## 13 arr_del15               0.0154
## 14 arr_delay_group         0.0154
## 15 actual_elapsed_time     0.0154
## 16 air_time                0.0154
## 17 wheels_on               0.0138
## 18 taxi_in                 0.0138
## 19 arr_time                0.0138
## 20 taxi_out                0.0137
## 21 wheels_off              0.0137
## 22 dep_time                0.0133
## 23 dep_delay               0.0133
## 24 dep_delay_new           0.0133
## 25 dep_del15               0.0133
## 26 dep_delay_group         0.0133
## 27 tail_num                0.00158
```

```r
# Missingness among key pre-departure predictors
key_predictors <- c(
 "dep_del15", "crs_dep_time", "sched_dep_hour", "op_unique_carrier",
 "dest", "distance", "month", "day_of_week", "flights"
)

key_predictors <- intersect(key_predictors, names(train))

train |>
 summarise(across(all_of(key_predictors), ~mean(is.na(.)))) |>
 pivot_longer(everything(), names_to = "variable", values_to = "pct_missing") |>
 mutate(pct_missing = percent(pct_missing, accuracy = 0.1))
```

```
## # A tibble: 9 x 2
```

```
##   variable          pct_missing
##   <chr>             <chr>
## 1 dep_del15         1.3%
## 2 crs_dep_time      0.0%
## 3 sched_dep_hour    0.0%
## 4 op_unique_carrier 0.0%
## 5 dest              0.0%
## 6 distance          0.0%
## 7 month             0.0%
## 8 day_of_week       0.0%
## 9 flights           0.0%
```

```
# Check whether missing targets line up with likely cancellations
train |>
 mutate(cancelled_proxy = is.na(dep_time) & is.na(dep_del15)) |>
 count(cancelled_proxy, missing_target = is.na(dep_del15))
```

```
## # A tibble: 2 x 3
##   cancelled_proxy missing_target     n
##   <lgl>           <lgl>          <int>
## 1 FALSE           FALSE          70506
## 2 TRUE            TRUE             950
```

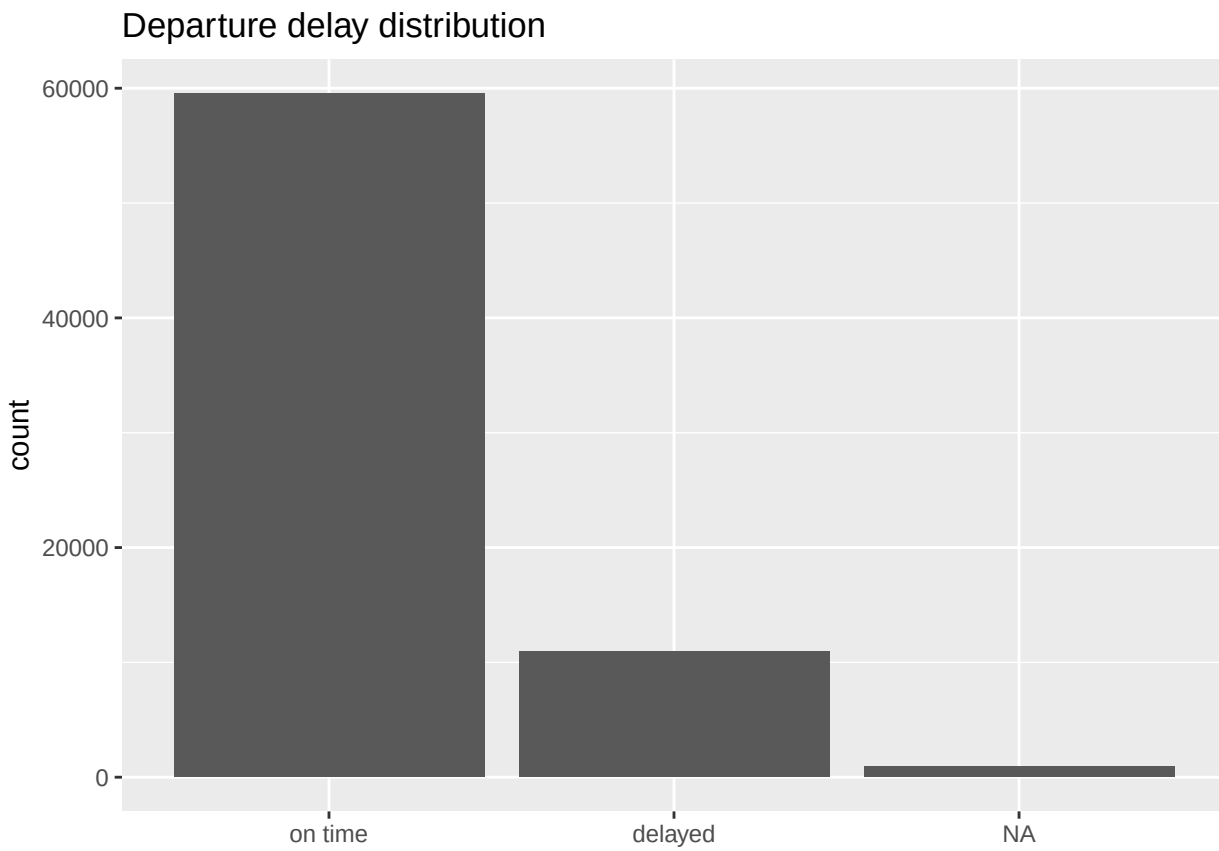
Missing target values align exactly with likely cancellations, so this is not unexplained missingness. We therefore treat these rows as cancelled flights and exclude them from the delay prediction task.

Departure Delay Distribution

```
# Target distribution
train |>
 count(dep_del15)
```

```
## # A tibble: 3 x 2
##   dep_del15     n
##       <dbl> <int>
## 1         0 59534
## 2         1 10972
## 3        NA   950
```

```
train |>
 ggplot(aes(x = factor(dep_del15, labels = c("on time", "delayed")))) +
 geom_bar() +
 labs(
 title = "Departure delay distribution",
 x = NULL,
 y = "count"
 )
```

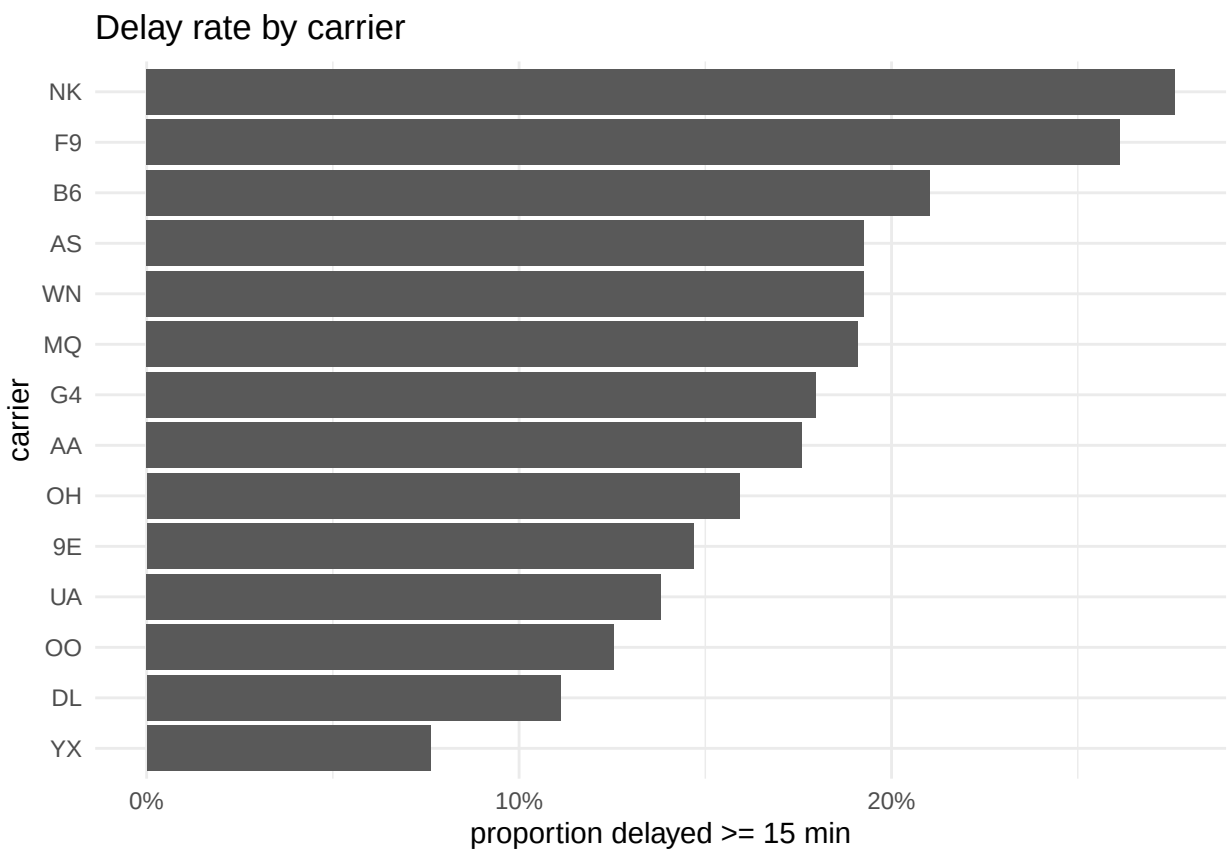


The classes are roughly 80/20 on-time versus delayed. This imbalance is worth keeping in mind, but it is not too concerning since our main evaluation metric is AUC rather than raw accuracy.

Delay Rate by Carrier

```
carrier_delay_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(op_unique_carrier) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(desc(delay_rate))

carrier_delay_tbl |>
 ggplot(aes(x = reorder(op_unique_carrier, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by carrier",
 x = "carrier",
 y = "proportion delayed >= 15 min"
 ) +
```

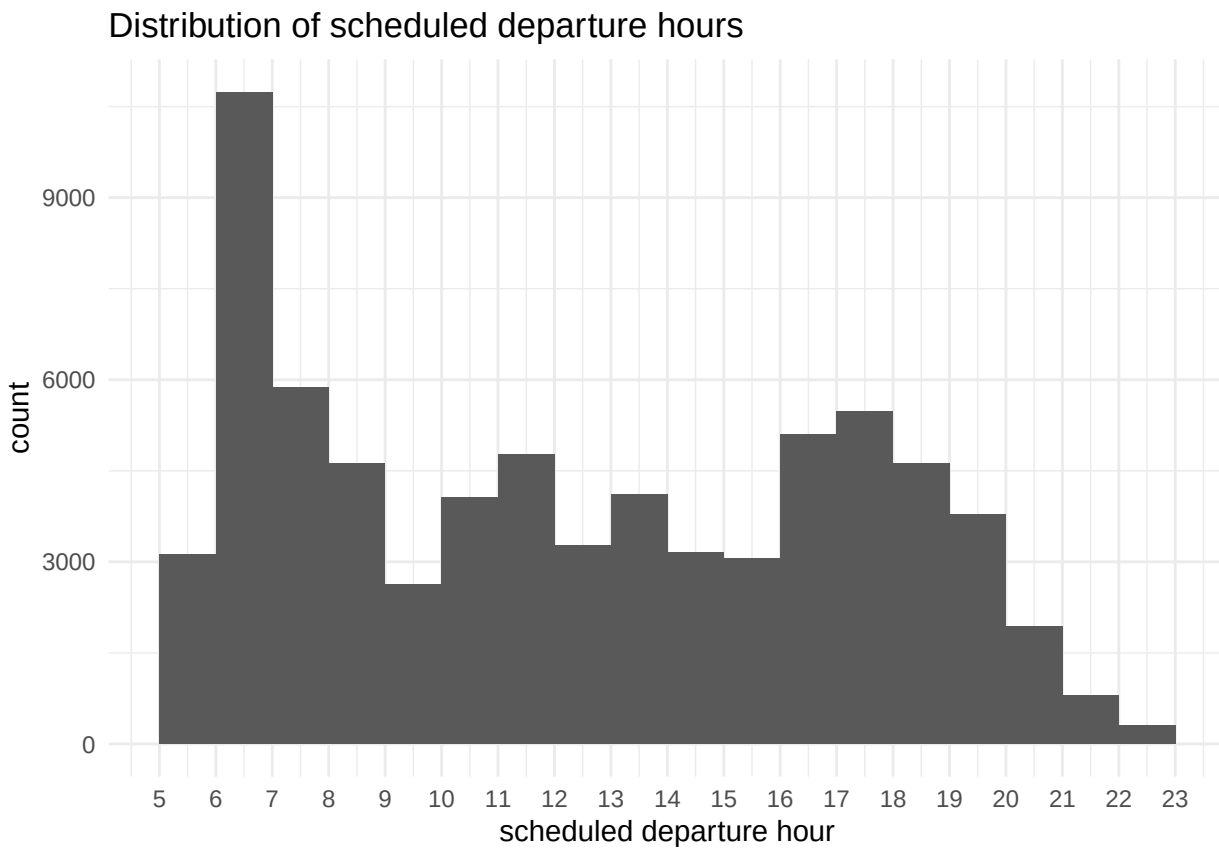
```
theme_minimal()
```



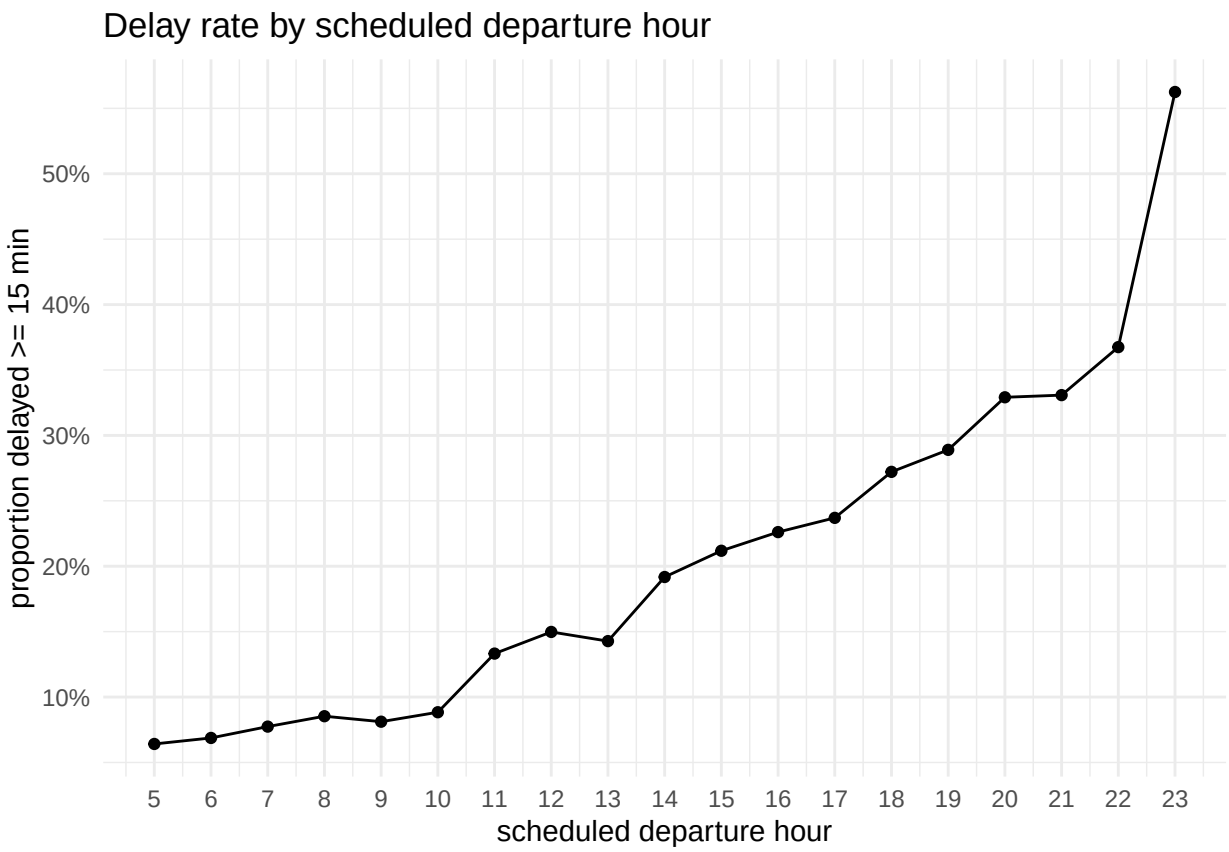
Delay rates vary meaningfully across carriers, so carrier should be a useful predictor. At the same time, some carriers have much lower flight counts at PIT than others, so raw group means will be noisy. That motivates using smoothed historical delay rates later in feature engineering instead of unsmoothed averages.

Delay Rate by Scheduled Departure Hour

```
# Distribution of scheduled departure times
train |>
 ggplot(aes(x = sched_dep_hour)) +
 geom_histogram(binwidth = 1, boundary = 0, closed = "left") +
 scale_x_continuous(breaks = 0:23) +
 labs(
 title = "Distribution of scheduled departure hours",
 x = "scheduled departure hour",
 y = "count"
 ) +
 theme_minimal()
```




```
hour_delay_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 )

hour_delay_tbl |>
 ggplot(aes(x = sched_dep_hour, y = delay_rate)) +
 geom_line() +
 geom_point() +
 scale_x_continuous(breaks = 0:23) +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by scheduled departure hour",
 x = "scheduled departure hour",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

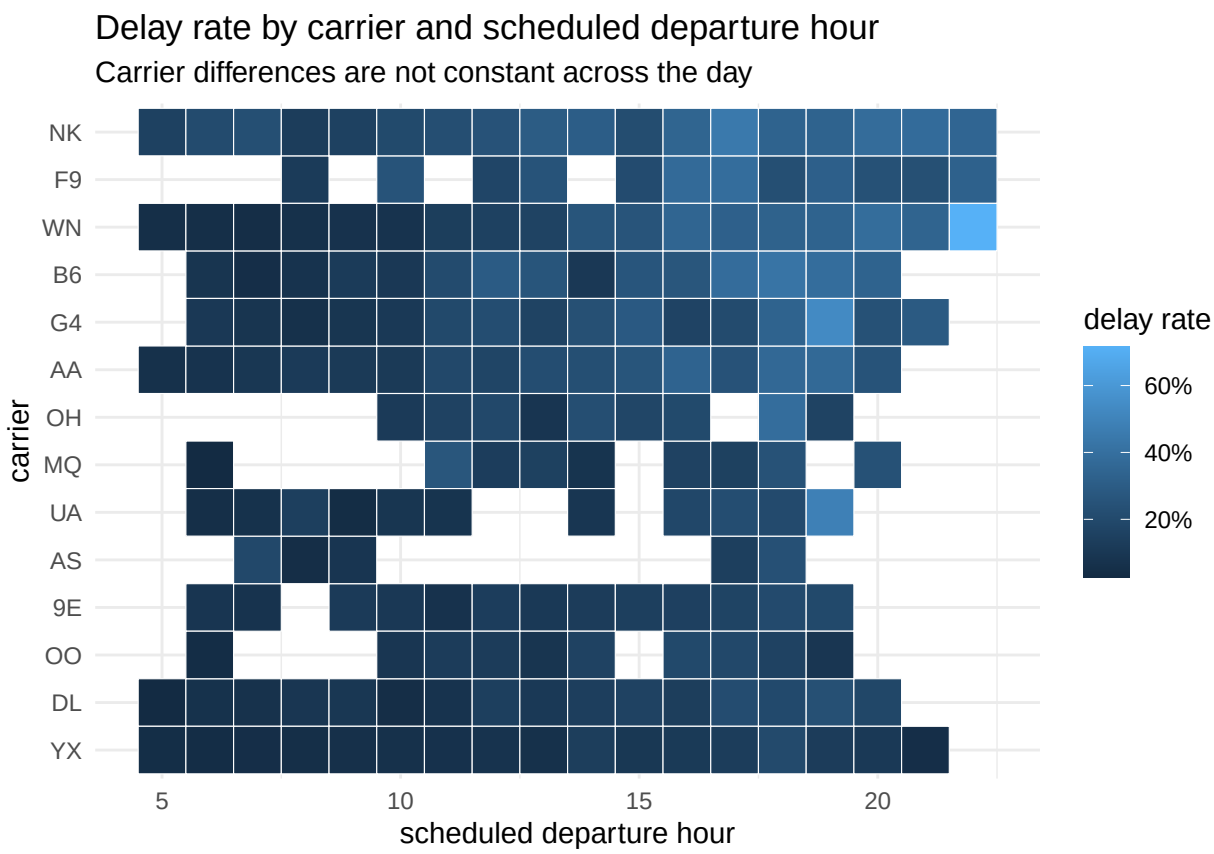


Delay rates rise steadily over the course of the day and are highest in the evening. This fits the idea of cascading delays: early flights start the day with fewer disruptions from previous legs, while later flights are more exposed to effects from earlier problems. This makes scheduled departure hour one of the strongest single predictors in the dataset.

Carrier-by-Hour Interaction

```
carrier_hour_tbl <- train |>
 filter(!is.na(dep_del15), !is.na(sched_dep_hour)) |>
 group_by(op_unique_carrier, sched_dep_hour) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 filter(n >= 20)

carrier_hour_tbl |>
 ggplot(
 aes(
 x = sched_dep_hour,
 y = fct_reorder(op_unique_carrier, delay_rate, .fun = mean),
 fill = delay_rate
 )
 ) +
```

```r
geom_tile(color = "white") +
scale_fill_gradient(labels = percent_format(accuracy = 1)) +
labs(
title = "Delay rate by carrier and scheduled departure hour",
subtitle = "Carrier differences are not constant across the day",
x = "scheduled departure hour",
y = "carrier",
fill = "delay rate"
) +
theme_minimal()
```

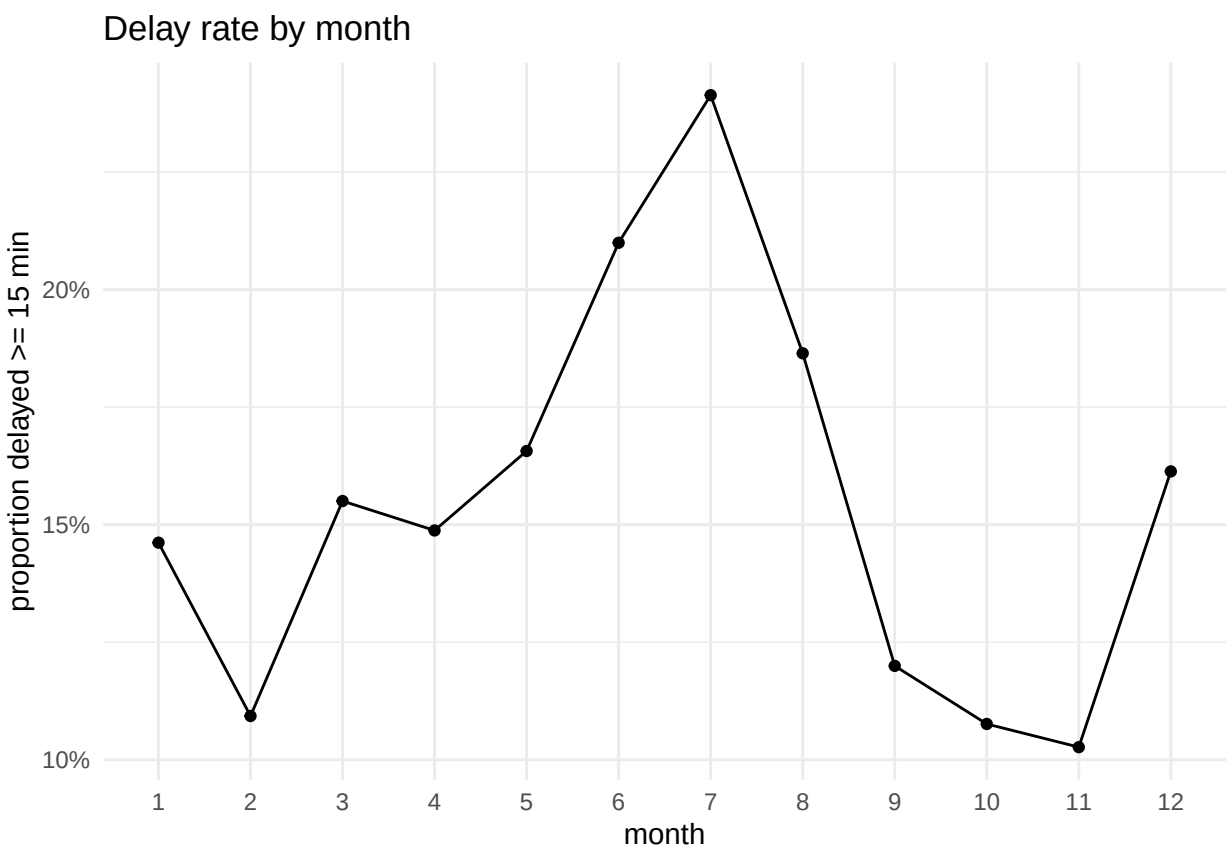


The carrier effect is not the same at every hour. Some carriers remain fairly stable across the day, while others become much more delay-prone in the evening. This suggests that a model with only main effects may miss meaningful structure, so either an explicit carrier-by-hour feature or a nonlinear model should help.

Seasonality and Calendar Effects

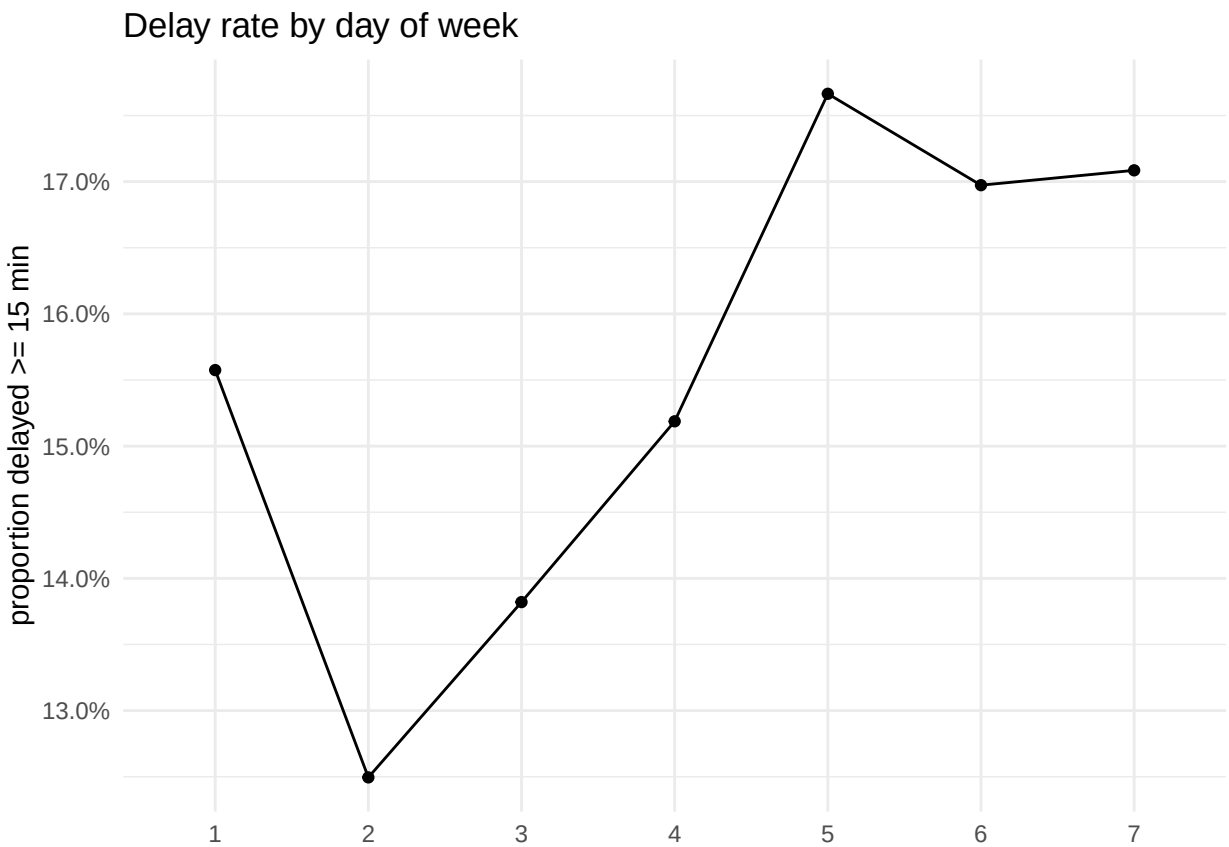
```r
# Delay rate by month
train |>
 filter(!is.na(dep_del15)) |>
 group_by(month) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
```

```r
) |>
ggplot(aes(x = month, y = delay_rate, group = 1)) +
geom_line() +
geom_point() +
scale_y_continuous(labels = percent_format()) +
labs(
title = "Delay rate by month",
x = "month",
y = "proportion delayed >= 15 min"
) +
theme_minimal()
```

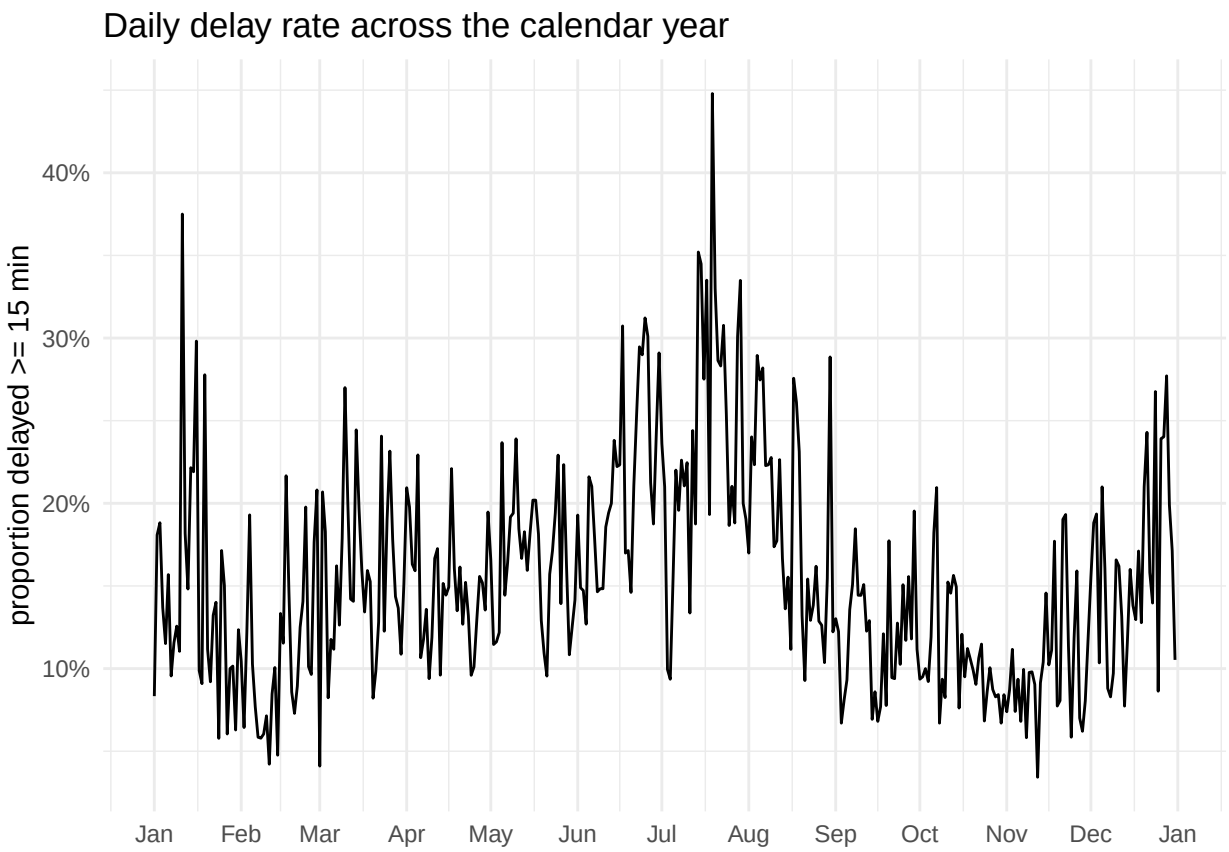



```r
# Delay rate by day of week
train |>
 filter(!is.na(dep_del15)) |>
 group_by(day_of_week) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = day_of_week, y = delay_rate, group = 1)) +
 geom_line() +
 geom_point() +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate by day of week",
```

```
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```




```
# Delay rate by calendar date, aligned to a common year for display
train |>
 filter(!is.na(dep_del15)) |>
 mutate(plot_date = make_date(year = 2010, month = as.integer(as.character(month)), day = day_of_month)
 group_by(plot_date) |>
 summarise(
 delay_rate = mean(dep_del15),
 .groups = "drop"
 ) |>
 ggplot(aes(x = plot_date, y = delay_rate)) +
 geom_line() +
 scale_x_date(date_labels = "%b", date_breaks = "1 month") +
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Daily delay rate across the calendar year",
 x = NULL,
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```

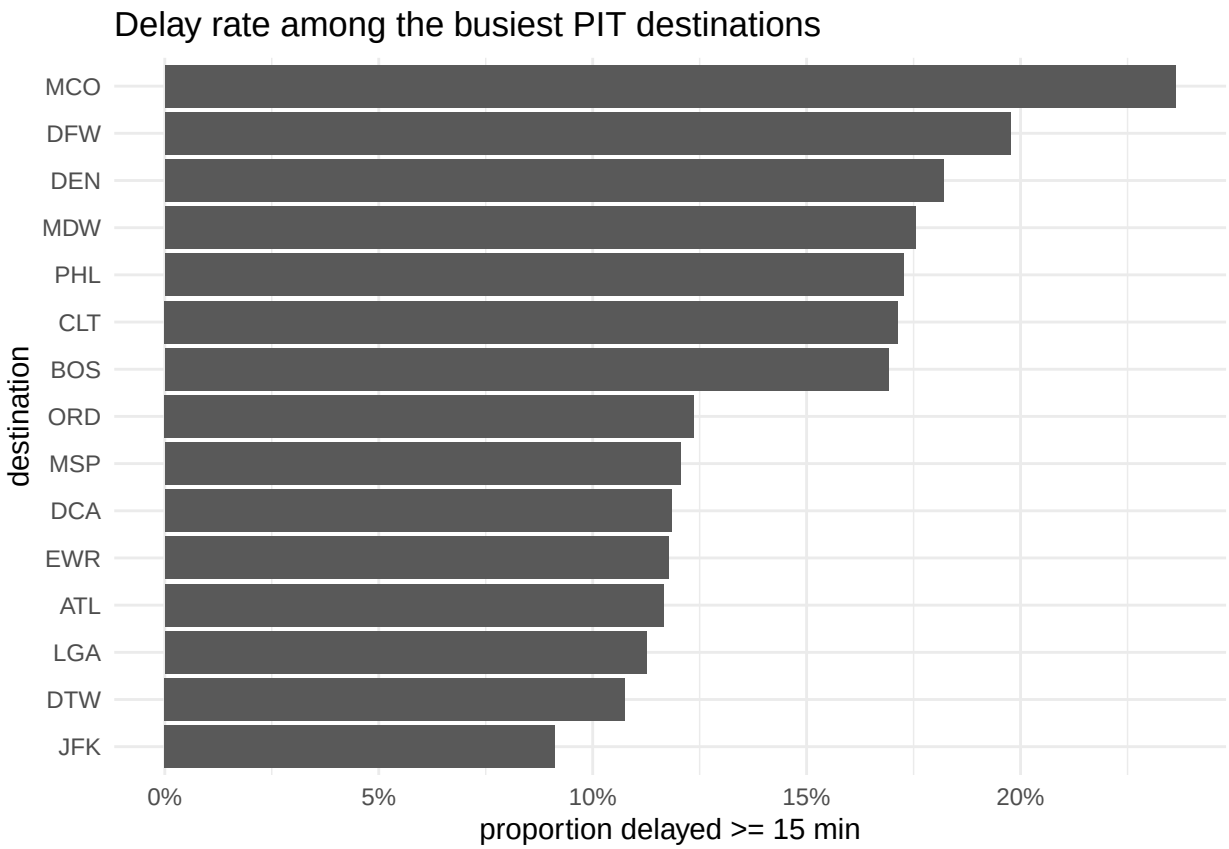


Delays are worse in the summer months and in December, which is consistent with heavier travel volume and more weather-related disruption. The day-of-week pattern is milder, but it still suggests that weekly operating patterns contain some signal. The daily calendar plot also shows that these seasonal peaks are not spread evenly across the month, which supports adding a simple calendar feature rather than relying only on month.

Destination and Distance Effects

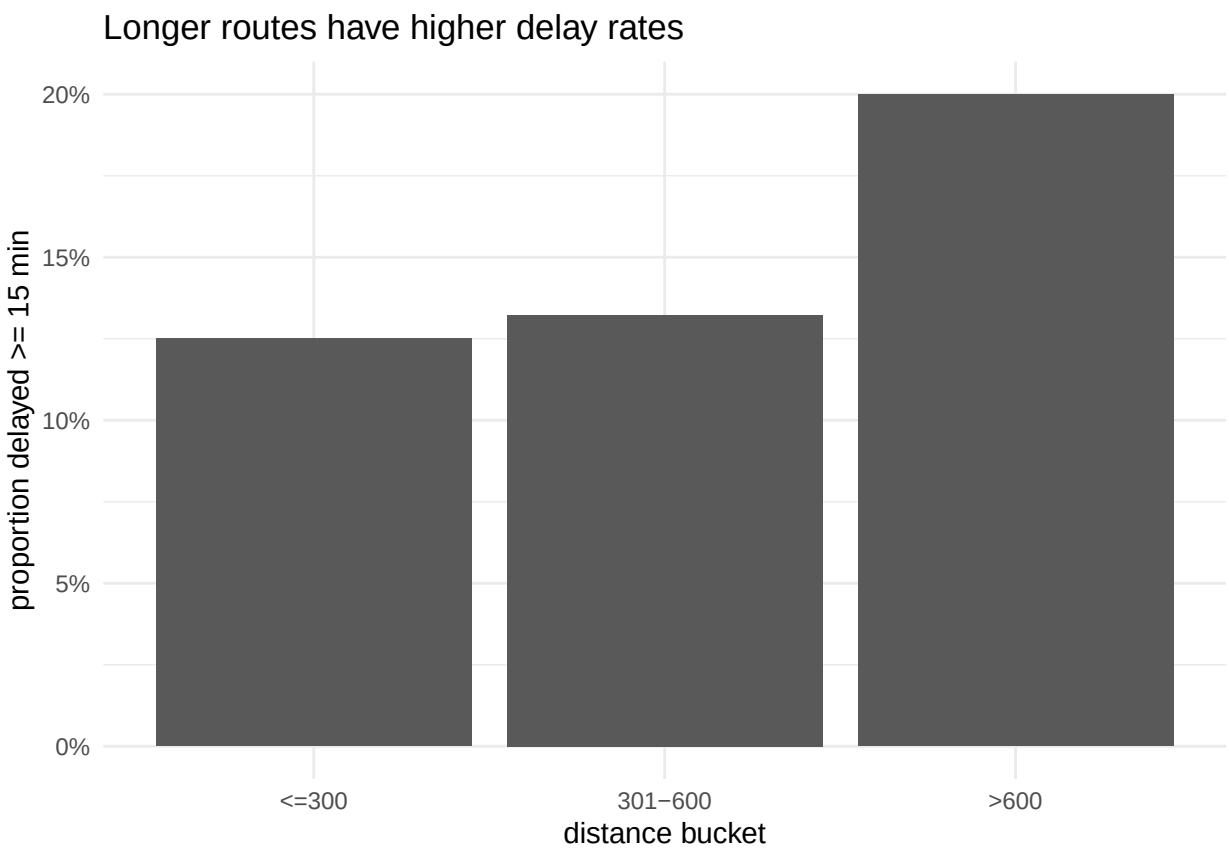
```
top_dests <- train |>
 count(dest, sort = TRUE) |>
 slice_head(n = 15) |>
 pull(dest)

train |>
 filter(!is.na(dep_del15), dest %in% top_dests) |>
 group_by(dest) |>
 summarise(
 delay_rate = mean(dep_del15),
 n = n(),
 .groups = "drop"
 ) |>
 arrange(delay_rate) |>
 ggplot(aes(x = reorder(dest, delay_rate), y = delay_rate)) +
 geom_col() +
 coord_flip() +
```

```
 scale_y_continuous(labels = percent_format()) +
 labs(
 title = "Delay rate among the busiest PIT destinations",
 x = "destination",
 y = "proportion delayed >= 15 min"
 ) +
 theme_minimal()
```




```
train <- train |>
 mutate(
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 distance_bucket = factor(
 distance_bucket,
 levels = c("<=300", "301-600", ">600")
 )
 )

train |>
 filter(!is.na(dep_del15), !is.na(distance_bucket)) |>
 group_by(distance_bucket) |>
 summarise(
 delay_rate = mean(dep_del15),
```

```
n = n(),
.groups = "drop"
) |>
ggplot(aes(x = distance_bucket, y = delay_rate)) +
geom_col() +
scale_y_continuous(labels = percent_format()) +
labs(
title = "Longer routes have higher delay rates",
x = "distance bucket",
y = "proportion delayed >= 15 min"
) +
theme_minimal()
```

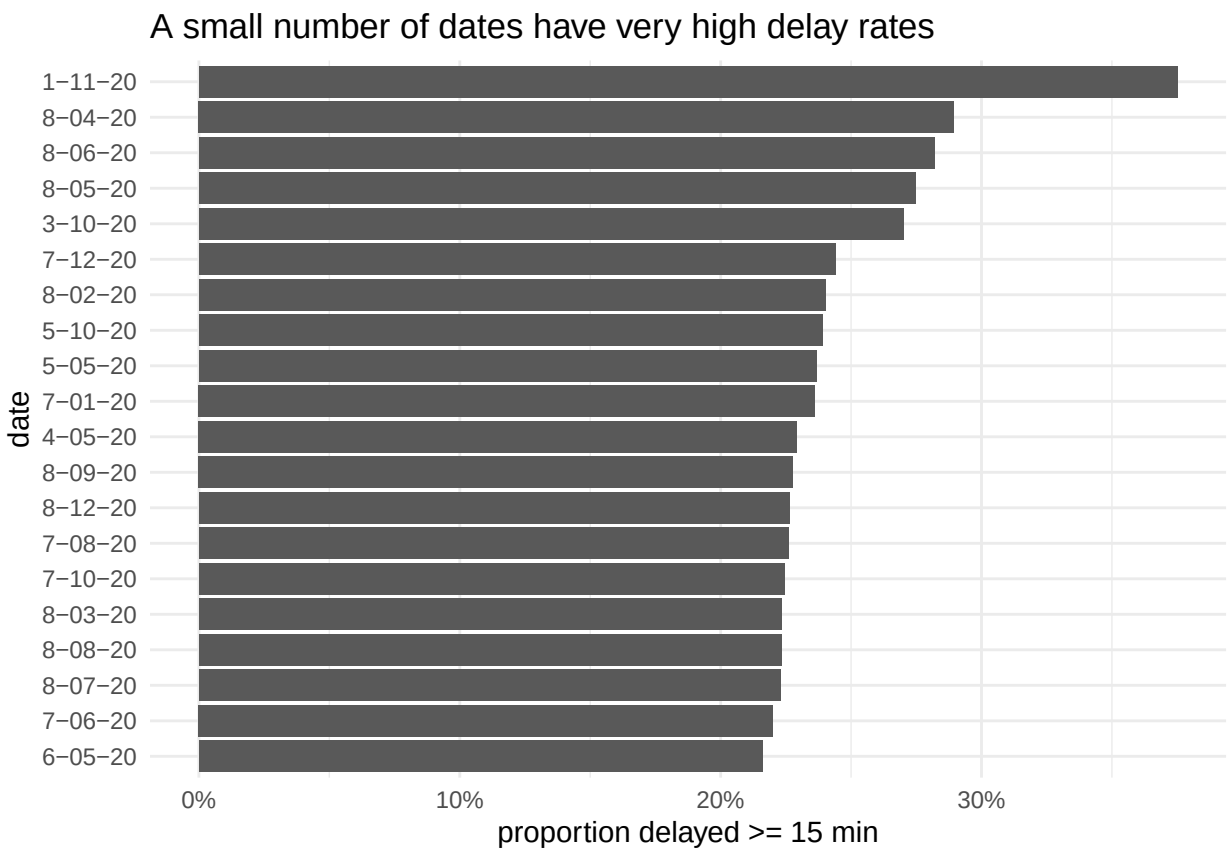


Delay rates differ across destinations, even among the busiest PIT routes, which suggests that route context adds information beyond carrier and time of day alone. Longer routes also show higher delay rates on average, so distance should remain in the model and may be easier to capture through both a raw continuous version and a bucketed factor version

High-Delay Days

```
bad_days_tbl <- train |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 delay_rate = mean(dep_del15),
```

```
  n = n(),
  .groups = "drop"
  ) |>
  filter(n >= 10) |>
  arrange(desc(delay_rate))

bad_days_tbl |>
  slice_head(n = 20) |>
  ggplot(aes(x = reorder(as.character(fl_date), delay_rate), y = delay_rate)) +
  geom_col() +
  coord_flip() +
  scale_y_continuous(labels = percent_format()) +
  labs(
  title = "A small number of dates have very high delay rates",
  x = "date",
  y = "proportion delayed >= 15 min"
  ) +
  theme_minimal()
```



A small number of days have unusually high delay rates, which suggests that there are broader operating shocks affecting many flights at once. That motivates adding dynamic same-day features based on earlier visible flights, along with a historical calendar-day disruption feature learned from the training data.

Feature Engineering

The EDA suggests that delay risk depends on a mix of stable historical tendencies and day-specific operating conditions. Based on those patterns, we engineer features that capture time-of-day effects, carrier and route history, route distance, seasonality, and same-day disruption.

Basic Features

```
add_basic_features <- function(df) {
 df |>
 mutate(
 peak_season = if_else(as.integer(as.character(month)) %in% c(6, 7, 8, 12), 1, 0),
 is_evening = if_else(sched_dep_hour >= 18, 1, 0),
 is_morning = if_else(sched_dep_hour <= 8, 1, 0),
 distance_bucket = case_when(
 distance > 600 ~ ">600",
 distance > 300 ~ "301-600",
 TRUE ~ "<=300"
 ),
 carrier_hour = paste(op_unique_carrier, sched_dep_hour, sep = "_")
 )
}

train <- add_basic_features(train)
test_visible <- add_basic_features(test_visible)
test_guess <- add_basic_features(test_guess)
```

These features directly reflect the strongest patterns from the EDA. In particular, `peak_season`, the morning and evening indicators, the distance bucket, and the carrier-by-hour combination all transform observed structure in the data into modeling variables.

Smoothed Historical Delay Rates

Raw group means are noisy, especially for carriers and routes with few flights at PIT. We use James-Stein style shrinkage: each group's rate is a weighted blend of its observed mean and the global delay rate, with the weight determined by how many observations the group has. Groups with lots of data are trusted; groups with little data are pulled toward the global mean.

```
global_delay_rate <- mean(train$dep_del15, na.rm = TRUE)

make_smoothed_rates <- function(df, group_var, target = "dep_del15", m = 100) {
 group_sym <- rlang::sym(group_var)
 target_sym <- rlang::sym(target)

 df |>
 filter(!is.na(!!target_sym)) |>
 group_by(!!group_sym) |>
 summarise(
 n = n(),
 mean_target = mean(!!target_sym),
```

```r
 .groups = "drop"
 ) |>
 mutate(
 smoothed_rate = (n * mean_target + m * global_delay_rate) / (n + m)
 ) |>
 select(!!group_sym, smoothed_rate, n)
}
```

```r
# Route = carrier + dest; more specific than either alone
train <- train |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_visible <- test_visible |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))
test_guess <- test_guess |> mutate(route = paste(op_unique_carrier, dest, sep = "_"))

# Carrier and dest rates computed on full train - stable enough that leakage is minimal
carrier_rates <- make_smoothed_rates(train, "op_unique_carrier", m = 200) |>
 rename(carrier_delay_rate_smooth = smoothed_rate, carrier_n = n)

dest_rates <- make_smoothed_rates(train, "dest", m = 200) |>
 rename(dest_delay_rate_smooth = smoothed_rate, dest_n = n)
```

```r
# Out-of-fold (OOF) encoding for fine-grained features
# carrier_hour, route, and flight_num are granular enough that computing them on the full
# training set leaks label info into val rows. OOF fixes this: each row's rate is computed
# from the other 4 folds only. Test sets use full-train lookups since their labels are never seen.
set.seed(42)
n_folds <- 5
fold_id <- sample(rep(1:n_folds, length.out = nrow(train)))

make_oof_rates <- function(df, group_var, target = "dep_del15", m = 100) {
  group_sym  <- rlang::sym(group_var)
  target_sym <- rlang::sym(target)
  out_col    <- paste0(group_var, "_oof_rate")
  global     <- mean(df[[target]], na.rm = TRUE)

  result <- numeric(nrow(df))
  for (k in 1:n_folds) {
    in_fold  <- which(fold_id == k)
    out_fold <- which(fold_id != k)
    lookup <- df[out_fold, ] |>
      filter(!is.na(!!target_sym)) |>
      group_by(!!group_sym) |>
      summarise(n = n(), mean_t = mean(!!target_sym), .groups = "drop") |>
      mutate(rate = (n * mean_t + m * global) / (n + m))
    merged <- df[in_fold, ] |>
      select(!!group_sym) |>
      left_join(lookup |> select(!!group_sym, rate), by = group_var)
    result[in_fold] <- coalesce(merged$rate, global)
  }
  result
}

train$carrier_hour_delay_rate_smooth     <- make_oof_rates(train, "carrier_hour",  m = 100)
train$route_delay_rate_smooth            <- make_oof_rates(train, "route",         m = 150)
```

```r
train$flight_num_delay_rate_smooth       <- make_oof_rates(train, "flights",       m = 150)

# Week-of-year rate replaces a raw calendar-day rate - only ~2 obs per calendar day
# with 2 years of data, which is too noisy. Week of year gives ~14 obs per cell.
train <- train |> mutate(week_of_year = as.character(week(fl_date)))
test_visible <- test_visible |> mutate(week_of_year = as.character(week(fl_date)))
test_guess   <- test_guess   |> mutate(week_of_year = as.character(week(fl_date)))
train$week_delay_rate_smooth <- make_oof_rates(train, "week_of_year", m = 50)
train$avg_delay_rate_for_calendar_day <- train$week_delay_rate_smooth  # kept for high_disruption deriv

# Full-train lookups for test sets
carrier_hour_rates <- make_smoothed_rates(train, "carrier_hour", m = 100) |>
  rename(carrier_hour_delay_rate_smooth = smoothed_rate, carrier_hour_n = n)

flight_num_rates <- make_smoothed_rates(train, "flights", m = 150) |>
  rename(flight_num_delay_rate_smooth = smoothed_rate, flight_num_n = n)

route_rates <- make_smoothed_rates(train, "route", m = 150) |>
  rename(route_delay_rate_smooth = smoothed_rate, route_n = n)

week_rates <- make_smoothed_rates(train, "week_of_year", m = 50) |>
  rename(week_delay_rate_smooth = smoothed_rate, week_n = n)

bad_day_lookup <- train |>
  filter(!is.na(dep_del15)) |>
  group_by(week_of_year) |>
  summarise(avg_delay_rate_for_calendar_day = mean(dep_del15), .groups = "drop")
```

Apply Historical Features to Train and Test

```r
# Train rows get OOF-encoded rates (already attached above); only carrier/dest joined here.
# Test rows get full-train lookups for all rate features.
apply_rate_features <- function(df, is_train = FALSE) {
  df <- df |>
    left_join(carrier_rates, by = "op_unique_carrier") |>
    left_join(dest_rates, by = "dest") |>
    mutate(
      carrier_delay_rate_smooth = coalesce(carrier_delay_rate_smooth, global_delay_rate),
      dest_delay_rate_smooth    = coalesce(dest_delay_rate_smooth,    global_delay_rate)
    )
  if (!is_train) {
    df <- df |>
      left_join(carrier_hour_rates, by = "carrier_hour") |>
      left_join(flight_num_rates,   by = "flights") |>
      left_join(route_rates,        by = "route") |>
      left_join(week_rates,         by = "week_of_year") |>
      left_join(bad_day_lookup,     by = "week_of_year") |>
      mutate(
        carrier_hour_delay_rate_smooth  = coalesce(carrier_hour_delay_rate_smooth,  global_delay_rate),
        flight_num_delay_rate_smooth    = coalesce(flight_num_delay_rate_smooth,    global_delay_rate),
        route_delay_rate_smooth         = coalesce(route_delay_rate_smooth,         global_delay_rate),
```

```r
        week_delay_rate_smooth          = coalesce(week_delay_rate_smooth,          global_delay_rate),
        avg_delay_rate_for_calendar_day = coalesce(avg_delay_rate_for_calendar_day, global_delay_rate)
      )
  }
  df
}

train_fe        <- apply_rate_features(train,        is_train = TRUE)
test_visible_fe <- apply_rate_features(test_visible, is_train = FALSE)
test_guess_fe   <- apply_rate_features(test_guess,   is_train = FALSE)
```

Same-Day Disruption Features

```r
# For each flight, count how many flights departed earlier that day and what fraction were delayed.
# This captures real-time system disruption - if the morning has been chaotic, later flights will be to
add_running_day_features <- function(df) {
 df |>
 arrange(fl_date, crs_dep_time) |>
 group_by(fl_date) |>
 mutate(
 before_today = row_number() - 1,
 delays_before_today = lag(cumsum(replace_na(dep_del15, 0)), default = 0),
 prior_same_day_delay_rate = if_else(
 before_today > 0,
 delays_before_today / before_today,
 0
 )
 ) |>
 ungroup()
}

train_fe <- add_running_day_features(train_fe)
test_visible_fe <- add_running_day_features(test_visible_fe)
test_guess_fe <- add_running_day_features(test_guess_fe)
```

This feature captures how disrupted the system already is earlier in the day. It is especially useful because the prediction setup gives visibility into earlier flights on the same date.

```r
# For the guess flights, we can't observe their own labels, so we use summary stats
# from the visible flights earlier that day as a proxy for same-day disruption.
test_visible_day_summary <- test_visible_fe |>
 filter(!is.na(dep_del15)) |>
 group_by(fl_date) |>
 summarise(
 visible_count = n(),
 visible_delay_rate = mean(dep_del15),
 .groups = "drop"
 )

test_guess_fe <- test_guess_fe |>
 left_join(test_visible_day_summary, by = "fl_date") |>
```

```r
mutate(
visible_count = coalesce(visible_count, 0),
visible_delay_rate = coalesce(visible_delay_rate, global_delay_rate)
)
```

For the guess set, we cannot directly use the unknown future labels. Instead, we summarize the flights that are already visible on that date and merge those day-level summaries into the flights we need to predict.

Carrier and Destination Encodings

Raw `op_unique_carrier` and `dest` factors can't go directly into the model because 2025 may contain carriers or destinations not seen in training, which would produce NA columns in the model matrix. Instead we use three stable encodings: a carrier size bucket (large/medium/small by flight count), a geographic region grouping for destinations, and a hub airport flag.

```r
# Carrier size by flight volume at PIT - unseen 2025 carriers default to "small"
carrier_size <- train |>
  count(op_unique_carrier) |>
  mutate(carrier_size = case_when(
    n >= 5000 ~ "large",
    n >= 1000 ~ "medium",
    TRUE      ~ "small"
  )) |>
  select(op_unique_carrier, carrier_size)

hub_airports <- c("ATL", "ORD", "DFW", "DEN", "CLT", "JFK", "LAX",
                  "LGA", "EWR", "SFO", "SEA", "MIA", "BOS", "MSP", "DTW", "PHL")

# Regional grouping for destinations - stable across years, unseen airports default to "other"
dest_region <- train |>
  distinct(dest) |>
  mutate(dest_region = case_when(
    dest %in% c("BOS", "JFK", "LGA", "EWR", "PHL", "DCA", "IAD", "BWI") ~ "northeast",
    dest %in% c("ATL", "CLT", "MIA", "TPA", "MCO", "RDU", "BNA", "MEM") ~ "south",
    dest %in% c("ORD", "MDW", "DTW", "MSP", "CLE", "CMH", "IND", "MKE") ~ "midwest",
    dest %in% c("DFW", "DAL", "IAH", "HOU", "MSY", "SAT", "AUS")        ~ "southwest",
    dest %in% c("DEN", "SLC", "PHX", "LAS", "ABQ")                      ~ "mountain",
    dest %in% c("LAX", "SFO", "SEA", "PDX", "SAN", "SJC", "OAK")       ~ "west",
    TRUE ~ "other"
  ))

add_carrier_dest_features <- function(df) {
  df |>
    left_join(carrier_size, by = "op_unique_carrier") |>
    left_join(dest_region, by = "dest") |>
    mutate(
      carrier_size = coalesce(carrier_size, "small"),   # unseen carriers → small
      dest_region  = coalesce(dest_region, "other"),    # unseen dests → other
      is_hub_dest  = if_else(dest %in% hub_airports, 1L, 0L),
      carrier_size = factor(carrier_size, levels = c("small", "medium", "large")),
      dest_region  = factor(dest_region),
      # derive binary flag from week rate (more stable than calendar-day rate)
```

```r
      high_disruption_calendar_day = if_else(
        week_delay_rate_smooth >= quantile(week_delay_rate_smooth, 0.90, na.rm = TRUE),
        1L, 0L
      )
    )
}

train_fe        <- add_carrier_dest_features(train_fe)
test_visible_fe <- add_carrier_dest_features(test_visible_fe)
test_guess_fe   <- add_carrier_dest_features(test_guess_fe)
```

Final Modeling Data Frames

```r
model_vars <- c(
  "dep_del15",
  "sched_dep_hour",
  "carrier_size",
  "dest_region",
  "is_hub_dest",
  "distance",
  "distance_bucket",
  "month",
  "day_of_week",
  "peak_season",
  "is_evening",
  "is_morning",
  "carrier_delay_rate_smooth",
  "dest_delay_rate_smooth",
  # carrier_hour_delay_rate_smooth removed - redundant given carrier_rate + sched_dep_hour,
  # and too granular to encode reliably even with OOF (dominated feature importance pathologically)
  "flight_num_delay_rate_smooth",
  "route_delay_rate_smooth",
  "prior_same_day_delay_rate",
  "before_today",
  "visible_delay_rate",
  "week_delay_rate_smooth",        # replaces avg_delay_rate_for_calendar_day (too noisy: ~2 obs/cell)
  "high_disruption_calendar_day"
)

train_model_df <- train_fe |>
 select(any_of(model_vars)) |>
 filter(!is.na(dep_del15))

test_visible_model_df <- test_visible_fe |>
 select(any_of(model_vars))

test_guess_model_df <- test_guess_fe |>
 select(any_of(model_vars))
```

At this point, `train_model_df` contains the response and engineered predictors for model fitting, while `test_guess_model_df` contains the same predictors for the flights we need to score. The final feature set

reflects the main conclusions from the EDA: time of day is a major driver, carrier, destination, route are important, and same-day disruption adds useful information.

Modelling

Setup

```r
# Helper to align model matrices between train and test so factor levels always match.
# Adds zero columns for any level in train not seen in test, drops extras.
make_aligned_matrix <- function(formula, train_df, test_df) {
  x_train <- model.matrix(formula, data = train_df)[, -1, drop = FALSE]
  x_test  <- model.matrix(formula, data = test_df)[, -1, drop = FALSE]

  missing_cols <- setdiff(colnames(x_train), colnames(x_test))
  if (length(missing_cols) > 0) {
    for (col in missing_cols) {
      x_test <- cbind(x_test, 0)
      colnames(x_test)[ncol(x_test)] <- col
    }
  }

  extra_cols <- setdiff(colnames(x_test), colnames(x_train))
  if (length(extra_cols) > 0) {
    x_test <- x_test[, !(colnames(x_test) %in% extra_cols), drop = FALSE]
  }

  x_test <- x_test[, colnames(x_train), drop = FALSE]

  list(
    x_train = x_train,
    x_test = x_test
  )
}
```

```r
# Helper to align factor levels between a new data frame and the training split.
# Needed for randomForest which requires the same factor levels at predict time.
align_to_training <- function(new_df, train_df) {
  common_cols <- intersect(names(train_df), names(new_df))

  for (col in common_cols) {
    # align factor columns
    if (is.factor(train_df[[col]])) {
      new_df[[col]] <- factor(as.character(new_df[[col]]),
                              levels = levels(train_df[[col]]))
    }

    # align numeric/integer columns
    if (is.numeric(train_df[[col]]) && !is.numeric(new_df[[col]])) {
      new_df[[col]] <- as.numeric(new_df[[col]])
    }
```

```r
    if (is.integer(train_df[[col]]) && !is.integer(new_df[[col]])) {
      new_df[[col]] <- as.integer(new_df[[col]])
    }
  }

  new_df
}
```

```r
# 80/20 train/val split for internal model evaluation
set.seed(100)

n <- nrow(train_model_df)
train_idx <- sample(1:n, size = 0.8 * n)

train_split <- train_model_df[train_idx, ]
test_split  <- train_model_df[-train_idx, ]
```

```r
# AUC and accuracy tracker - appends one row per model
compute_auc <- function(auc_df = NULL, train_y, train_prob,
                        test_y, test_prob, model_name) {
  auc_train <- roc(train_y, train_prob) |> auc()
  auc_test <- roc(test_y,  test_prob)  |> auc()

  accur_train_0.5 <- mean(train_y == (train_prob > 0.5))
  accur_test_0.5 <- mean(test_y  == (test_prob  > 0.5))

  accur_train_0.6 <- mean(train_y == (train_prob > 0.6))
  accur_test_0.6 <- mean(test_y  == (test_prob  > 0.6))

  df <- data.frame(
    model_name,
    auc_train,
    auc_test,
    accur_train_0.5,
    accur_test_0.5,
    accur_train_0.6,
    accur_test_0.6
  )

  if (is.null(auc_df)) {
    return(df)
  } else {
    return(rbind(auc_df, df))
  }
}
```

```r
y_train <- train_split$dep_del15
y_test  <- test_split$dep_del15

# Linear model matrix: poly(sched_dep_hour, 2) lets logistic/LASSO capture the curved hour effect
lm_mats <- make_aligned_matrix(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  train_split,
```

```r
  test_split
)

x_train_lm <- lm_mats$x_train
x_test_lm <- lm_mats$x_test

# XGBoost matrix: raw features only - XGBoost finds nonlinearity through splits, not poly terms
xgb_mats <- make_aligned_matrix(
  dep_del15 ~ .,
  train_split,
  test_split
)

x_train_xgb <- xgb_mats$x_train
x_test_xgb <- xgb_mats$x_test

# Full-train matrix used for XGBoost CV tuning (avoids overfitting to the 80/20 split)
x_full_xgb <- model.matrix(dep_del15 ~ ., data = train_model_df)[, -1, drop = FALSE]
y_full <- train_model_df$dep_del15
```

Logistic Regression

```r
log_reg <- glm(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  family = "binomial",
  data = train_split
)

train_prob <- predict(log_reg, type = "response", newdata = train_split)
test_prob  <- predict(log_reg, type = "response", newdata = test_split)

auc_df <- compute_auc(
  train_y = train_split$dep_del15,
  train_prob = train_prob,
  test_y = test_split$dep_del15,
  test_prob = test_prob,
  model_name = "Log Reg"
)
```

LASSO

```r
cv_lasso <- cv.glmnet(
  x_train_lm,
  y_train,
  family = "binomial",
  alpha = 1
)

train_prob <- predict(
```

```r
  cv_lasso,
  newx = x_train_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

test_prob <- predict(
  cv_lasso,
  newx = x_test_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Log Reg (LASSO)"
)
```

XGBoost (tuned)

XGBoost with default params and 100 rounds leaves a lot of AUC on the table. We tune the key parameters via cross-validation: `max_depth` controls tree complexity, `eta` controls shrinkage, and `subsample`/`colsample_bytree` add randomization to reduce overfitting. `nrounds` is selected by early stopping on the CV AUC curve.

```r
dtrain <- xgb.DMatrix(data = x_train_xgb, label = y_train)
dtest  <- xgb.DMatrix(data = x_test_xgb, label = y_test)
dfull  <- xgb.DMatrix(data = x_full_xgb, label = y_full)

xgb_grid <- expand.grid(
  max_depth        = c(3, 4, 5),
  eta              = c(0.05, 0.1),
  subsample        = c(0.7, 0.9),
  colsample_bytree = c(0.7, 0.9),
  min_child_weight = c(10, 30),
  gamma            = c(0, 1)
)

best_auc     <- 0
best_params  <- NULL
best_nrounds <- 100

for (i in seq_len(nrow(xgb_grid))) {
  params_i <- list(
    objective        = "binary:logistic",
    eval_metric      = "auc",
    max_depth        = xgb_grid$max_depth[i],
```

```r
    eta              = xgb_grid$eta[i],
    subsample        = xgb_grid$subsample[i],
    colsample_bytree = xgb_grid$colsample_bytree[i],
    min_child_weight = xgb_grid$min_child_weight[i],
    gamma            = xgb_grid$gamma[i]
  )

  cv_result <- xgb.cv(
    params = params_i,
    data = dfull,
    nrounds = 500,
    nfold = 5,
    early_stopping_rounds = 30,
    verbose = 0
  )

  nrounds_i <- cv_result$best_iteration
  if (is.null(nrounds_i) || length(nrounds_i) == 0) {
    nrounds_i <- which.max(cv_result$evaluation_log$test_auc_mean)
  }

  best_i <- cv_result$evaluation_log$test_auc_mean[nrounds_i]

  if (best_i > best_auc) {
    best_auc     <- best_i
    best_params  <- params_i
    best_nrounds <- nrounds_i
  }
}

cat("Best CV AUC:", round(best_auc, 4), "| nrounds:", best_nrounds, "\n")
```

```
## Best CV AUC: 0.7443 | nrounds: 145
```

```r
xgb_model <- xgb.train(
  params = best_params,
  data = dfull,
  nrounds = best_nrounds,
  verbose = 0
)

train_prob <- predict(xgb_model, dtrain)
test_prob  <- predict(xgb_model, dtest)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "XGBoost (tuned)"
)
```

Random Forest

```
train_split_rf <- train_split |>
  mutate(dep_del15 = as.factor(dep_del15))

test_split_rf <- test_split |>
  mutate(dep_del15 = as.factor(dep_del15))

rf_model <- randomForest(
  dep_del15 ~ .,
  data = train_split_rf,
  ntree = 300,
  importance = TRUE
)

train_prob <- predict(rf_model, newdata = train_split_rf, type = "prob")[, "1"]

test_split_rf_aligned <- align_to_training(test_split_rf, train_split_rf)

test_prob <- predict(rf_model, newdata = test_split_rf_aligned, type = "prob")[, "1"]

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Random Forest"
)
```

Balanced Random Forest

```
# Upweight the minority (delayed) class to counteract the 80/20 imbalance
class_counts <- table(train_split_rf$dep_del15)

class_weights <- as.numeric(
  sum(class_counts) / (length(class_counts) * class_counts)
)
names(class_weights) <- names(class_counts)

brf_model <- randomForest(
  dep_del15 ~ .,
  data = train_split_rf,
  ntree = 300,
  classwt = class_weights
)

train_prob <- predict(brf_model, newdata = train_split_rf, type = "prob")[, "1"]

test_split_rf_aligned <- align_to_training(test_split_rf, train_split_rf)
```

```r
test_prob <- predict(brf_model, newdata = test_split_rf_aligned, type = "prob")[, "1"]

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = y_train,
  train_prob = train_prob,
  test_y = y_test,
  test_prob = test_prob,
  model_name = "Balanced RF"
)
```

Ensembles

Simple averaging across all models dilutes the best performers. We compare: (1) the naive average, (2) a weighted average that upweights XGBoost, which typically dominates on tabular data, and (3) majority vote.

```r
preds_df <- data.frame(
  y_true = as.numeric(as.character(test_split$dep_del15)),
  log_reg = predict(log_reg, type = "response", newdata = test_split),
  lasso = predict(cv_lasso, newx = x_test_lm, s = "lambda.min", type = "response") |> as.vector(),
  xgboost = predict(xgb_model, dtest),
  rf = predict(rf_model,  newdata = test_split_rf, type = "prob")[, "1"],
  brf = predict(brf_model, newdata = test_split_rf, type = "prob")[, "1"]
)

# Simple average
preds_df$avg_prob <- rowMeans(preds_df[, c("log_reg", "lasso", "xgboost", "rf", "brf")])

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$avg_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$avg_prob,
  model_name = "Ensemble (Avg)"
)

# Weighted average: XGBoost gets more weight since it's the strongest model on this data
preds_df$weighted_prob <- with(
  preds_df,
  0.50 * xgboost +
    0.15 * rf +
    0.10 * brf +
    0.15 * lasso +
    0.10 * log_reg
)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$weighted_prob,
```

```r
  test_y = preds_df$y_true,
  test_prob = preds_df$weighted_prob,
  model_name = "Ensemble (Weighted)"
)

# Majority vote
pred_class <- preds_df[, c("log_reg", "lasso", "xgboost", "rf", "brf")] > 0.5
preds_df$majority_prob <- rowMeans(pred_class)

auc_df <- compute_auc(
  auc_df = auc_df,
  train_y = preds_df$y_true,
  train_prob = preds_df$majority_prob,
  test_y = preds_df$y_true,
  test_prob = preds_df$majority_prob,
  model_name = "Ensemble (Majority Vote)"
)
```

Results - 2025 Visible

```r
# AUC and accuracy on the 2025 visible set (the portion of test data where we know the true labels)
compute_vis_auc <- function(auc_df = NULL, y, prob, model_name) {
  auc <- roc(y, prob) |> auc()
  accuracy <- mean(y == (prob > 0.5), na.rm = TRUE)

  df <- data.frame(model_name, auc, accuracy)

  if (is.null(auc_df)) {
    return(df)
  } else {
    return(rbind(auc_df, df))
  }
}
```

```r
test_visible_model_df_eval <- test_visible_model_df |>
  filter(!is.na(dep_del15))

y_true <- as.numeric(as.character(test_visible_model_df_eval$dep_del15))
y_vis <- test_visible_model_df_eval$dep_del15

# Build aligned matrices using the original training split as the reference
vis_lm_mats <- make_aligned_matrix(
  dep_del15 ~ . + poly(sched_dep_hour, 2),
  train_split,
  test_visible_model_df_eval
)

x_vis_lm <- vis_lm_mats$x_test

vis_xgb_mats <- make_aligned_matrix(
  dep_del15 ~ .,
```

```r
  train_split,
  test_visible_model_df_eval
)

x_vis_xgb <- vis_xgb_mats$x_test
dvis <- xgb.DMatrix(data = x_vis_xgb, label = y_vis)

# Logistic / lasso / xgboost probabilities
log_reg_prob <- predict(log_reg, type = "response", newdata = test_visible_model_df_eval)

lasso_prob <- predict(
  cv_lasso,
  newx = x_vis_lm,
  s = "lambda.min",
  type = "response"
) |>
  as.vector()

xgb_prob <- predict(xgb_model, dvis)

# RF-compatible visible data (factor levels must match training)
test_visible_rf <- test_visible_model_df_eval |>
  mutate(dep_del15 = as.factor(dep_del15))

test_visible_rf <- align_to_training(test_visible_rf, train_split_rf)

rf_prob <- predict(
  rf_model,
  newdata = test_visible_rf,
  type = "prob"
)[, "1"]

brf_prob <- predict(
  brf_model,
  newdata = test_visible_rf,
  type = "prob"
)[, "1"]

avg_prob <- rowMeans(data.frame(log_reg_prob, lasso_prob, xgb_prob, rf_prob, brf_prob))

weighted_prob <- 0.50 * xgb_prob +
  0.15 * rf_prob +
  0.10 * brf_prob +
  0.15 * lasso_prob +
  0.10 * log_reg_prob

vis_auc <- compute_vis_auc(
  y = y_true,
  prob = log_reg_prob,
  model_name = "Log Reg"
)

vis_auc <- compute_vis_auc(
```

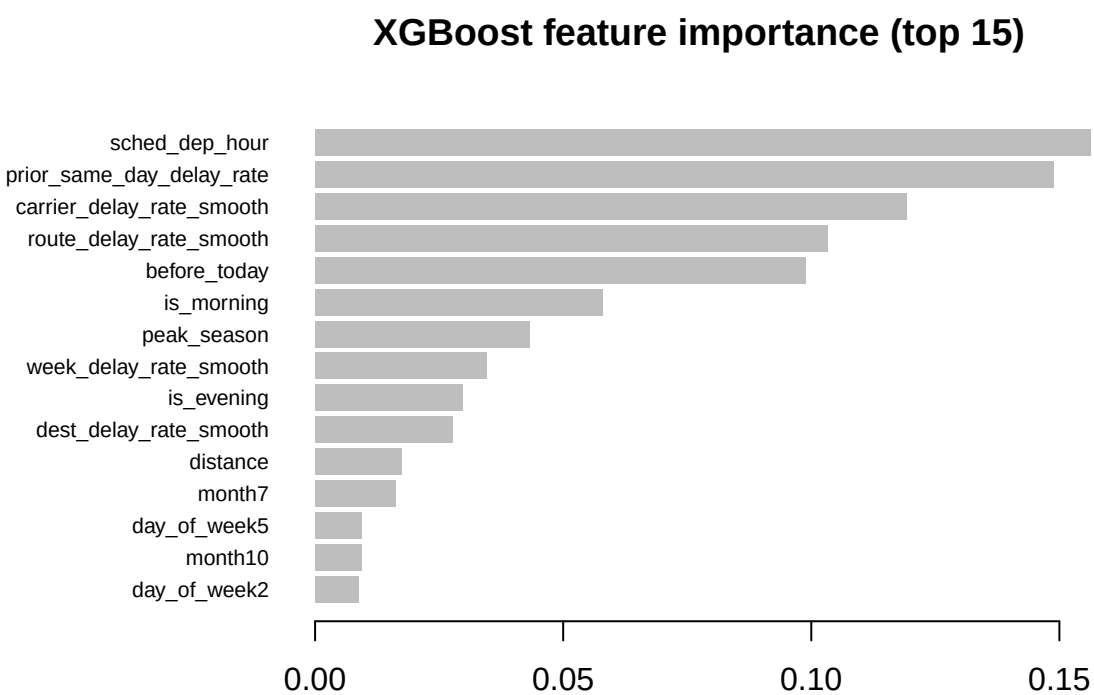
```r
  vis_auc,
  y = y_true,
  prob = lasso_prob,
  model_name = "LASSO"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = xgb_prob,
  model_name = "XGBoost (tuned)"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = rf_prob,
  model_name = "RF"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = brf_prob,
  model_name = "Balanced RF"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = avg_prob,
  model_name = "Ensemble (Avg)"
)

vis_auc <- compute_vis_auc(
  vis_auc,
  y = y_true,
  prob = weighted_prob,
  model_name = "Ensemble (Weighted)"
)
```

```r
vis_auc |> arrange(desc(auc))
```

```
##            model_name       auc  accuracy
## 1 Ensemble (Weighted) 0.6896571 0.8861746
## 2      Ensemble (Avg) 0.6875589 0.8862799
## 3     XGBoost (tuned) 0.6858221 0.8865958
## 4             Log Reg 0.6843659 0.8859640
## 5                  RF 0.6737819 0.8862799
## 6               LASSO 0.6608660 0.8859640
## 7         Balanced RF 0.6343557 0.8858587
```

```r
vis_auc |> arrange(desc(accuracy))
```

```
##            model_name       auc  accuracy
## 1     XGBoost (tuned) 0.6858221 0.8865958
## 2                  RF 0.6737819 0.8862799
## 3      Ensemble (Avg) 0.6875589 0.8862799
## 4 Ensemble (Weighted) 0.6896571 0.8861746
## 5             Log Reg 0.6843659 0.8859640
## 6               LASSO 0.6608660 0.8859640
## 7         Balanced RF 0.6343557 0.8858587
```

XGBoost Feature Importance

```r
xgb.importance(model = xgb_model) |>
  xgb.plot.importance(top_n = 15, main = "XGBoost feature importance (top 15)")
```